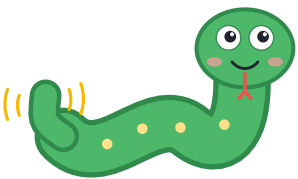
Lecture 4

Part 2: Decisions and Loops

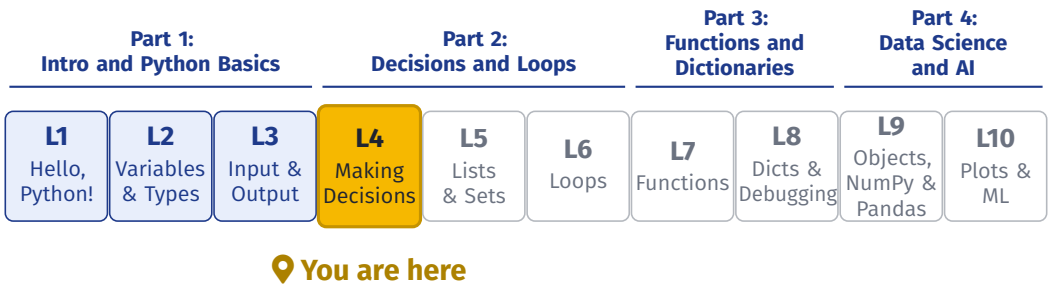
Making Decisions

if, elif, else, and/or/not, and a peek at the 0s and 1s inside



Dror Jacoby
Tel Aviv University

Where we are


Part 1:
Intro and Python Basics
Part 2:
Decisions and Loops
Part 3:
Functions and
Dictionaries
Part 4:
Data Science
and AI
L1
Hello,
Python!
L2
Variables
& Types
L3
Input &
Output
L4
Making
Decisions
L5
Lists
& Sets
L6
Loops
L7
Functions
L8
Dicts &
Debugging
L9
Objects,
NumPy &
Pandas
L10
Plots &
ML
You are here

You make a hundred decisions before 9 a.m.

- **If** it rains, take an umbrella

You make a hundred decisions before 9 a.m.

- **If** it rains, take an umbrella
- **If** the bus is late, grab a scooter

You make a hundred decisions before 9 a.m.

- **If** it rains, take an umbrella
- **If** the bus is late, grab a scooter
- **If** coffee costs over 20 ILS, make it at home

You make a hundred decisions before 9 a.m.

- **If** it rains, take an umbrella
- **If** the bus is late, grab a scooter
- **If** coffee costs over 20 ILS, make it at home
- Today the intern learns to decide, too



Today's plan

◎ By the end of today you can...

- Write programs that choose between several paths
- Combine conditions with `and`, `or`, `not`
- Explain how 0s and 1s store True and False



Today's outline

1 **if, elif, else**
Many options, the order of conditions, decisions inside decisions

2 **Combine conditions**
A quick reminder of comparisons, then `and`, `or`, `not`

3 **Under the hood: 0s and 1s**
Bits, `bin()`, and the logic gates inside every chip



Every topic: examples, **Try it!** and **Predict the output**. At the end: **Bug Zoo**, **AI Corner**, **Recap**.

Why today matters

Why learn this?

- Real programs react differently to different situations
- Rules like “students in the morning get 15% off” become code
- Every decision a computer makes comes down to 0s and 1s

Where you will see it

- Password checks and log-ins
- Traffic lights, thermostats, smart homes
- Spam filters and discounts at checkout



Topic 1

if, elif, else

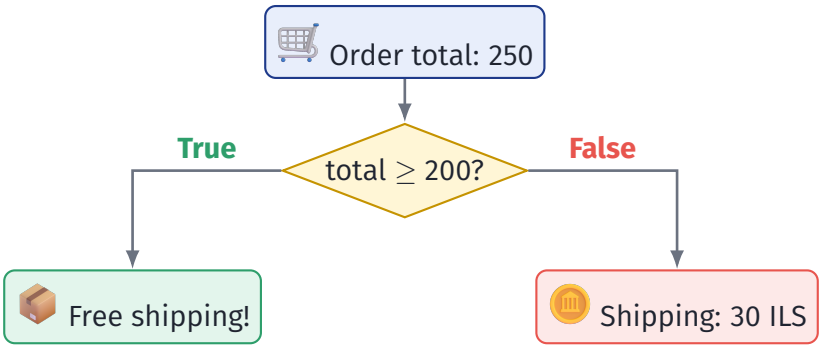
Lecture 4 • Making Decisions



TODAY

1. **if, elif, else**
2. Combine conditions
3. Under the hood: 0s and 1s

Reminder: if / else is a fork in the road



The intern checks the question, then walks down **one** road.

Indentation means “belongs to”

```
mood = "happy"
if mood == "happy":
    print("Dance")
print("Drink water")
```

```
mood = "tired"
if mood == "happy":
    print("Dance")
print("Drink water")
```

Indentation means “belongs to”

```
mood = "happy"
if mood == "happy":
    print("Dance")
print("Drink water")
```

```
mood = "tired"
if mood == "happy":
    print("Dance")
print("Drink water")
```

OUTPUT

```
Dance
Drink water
```

Indentation means “belongs to”

```
mood = "happy"
if mood == "happy":
    print("Dance")
print("Drink water")
```

```
OUTPUT
Dance
Drink water
```

```
mood = "tired"
if mood == "happy":
    print("Dance")
print("Drink water")
```

```
OUTPUT
Drink water
```

Indentation means “belongs to”

```
mood = "happy"
if mood == "happy":
    print("Dance")
print("Drink water")
```

```
OUTPUT
Dance
Drink water
```

```
mood = "tired"
if mood == "happy":
    print("Dance")
print("Drink water")
```

```
OUTPUT
Drink water
```

- Lines pushed right (4 spaces) belong to the `if`
- `print("Drink water")` is back on the left, so it always runs

More than two roads: elif

Your exam grade

- 90 or more: Excellent
- 75 or more: Very good
- 56 or more: Passed
- Otherwise: try again

- `elif` = “else, if…”
- Add as many as you need
- One `else` at the end (optional)



The grade calculator

L04_01_if_elif_else.ipynb

```
grade = 84
if grade >= 90:
    print("Excellent!")
elif grade >= 75:
    print("Very good")
elif grade >= 56:
    print("Passed")
else:
    print("Let's try again")
```

The grade calculator

L04_01_if_elif_else.ipynb

```
grade = 84
if grade >= 90:
    print("Excellent!")
elif grade >= 75:
    print("Very good")
elif grade >= 56:
    print("Passed")
else:
    print("Let's try again")
```

OUTPUT

```
Very good
```

The grade calculator

L04_01_if_elif_else.ipynb

```
grade = 84
if grade >= 90:
    print("Excellent!")
elif grade >= 75:
    print("Very good")
elif grade >= 56:
    print("Passed")
else:
    print("Let's try again")
```

OUTPUT

```
Very good
```



How the intern reads elif

```
84 >= 90 ?
```

```
84 >= 75 ?
```

```
84 >= 56 ?
```

```
else
```

How the intern reads elif

```
84 >= 90 ?
```

False → skip, check the next one

```
84 >= 75 ?
```

```
84 >= 56 ?
```

```
else
```

How the intern reads elif

```
84 >= 90 ?
```

False → skip, check the next one

```
84 >= 75 ?
```

True → print “Very good”

```
84 >= 56 ?
```

```
else
```

How the intern reads elif

`84 >= 90 ?`	False → skip, check the next one
`84 >= 75 ?`	**True** → print “Very good”
`84 >= 56 ?`	never checked
`else`	never checked

How the intern reads elif

`84 >= 90 ?`	False → skip, check the next one
`84 >= 75 ?`	**True** → print “Very good”
`84 >= 56 ?`	never checked
`else`	never checked

Top to bottom. The first True wins. Then stop.

Example: bus fare by age

```
age = int(input("Your age? "))
if age < 5:
    print("You ride for free!")
elif age >= 67:
    print("Senior ticket")
else:
    print("Regular ticket")
```

Simplified rules, just for practice.



Example: bus fare by age

```python
age = int(input("Your age? "))
if age < 5:
    print("You ride for free!")
elif age >= 67:
    print("Senior ticket")
else:
    print("Regular ticket")
```

OUTPUT

```
Your age? 72
Senior ticket
```

Simplified rules, just for practice.



Example: bus fare by age

```
age = int(input("Your age? "))
if age < 5:
    print("You ride for free!")
elif age >= 67:
    print("Senior ticket")
else:
    print("Regular ticket")
```

OUTPUT

```
Your age? 72
Senior ticket
```

OUTPUT

```
Your age? 3
You ride for free!
```

Simplified rules, just for practice.



Example: what should I wear?

```
temp = 14
if temp >= 28:
    print("T-shirt and sunscreen")
elif temp >= 18:
    print("Long sleeves")
else:
    print("Coat!")
```



Example: what should I wear?

```
temp = 14
if temp >= 28:
    print("T-shirt and sunscreen")
elif temp >= 18:
    print("Long sleeves")
else:
    print("Coat!")
```



OUTPUT

```
Coat!
```

Example: what should I wear?

```
temp = 14
if temp >= 28:
    print("T-shirt and sunscreen")
elif temp >= 18:
    print("Long sleeves")
else:
    print("Coat!")
```

OUTPUT

```
Coat!
```



Change `temp` to 30. What prints now?

Order matters!

```
grade = 95
if grade >= 56:
    print("Passed")
elif grade >= 90:
    print("Excellent!")
```

Order matters!

```
grade = 95
if grade >= 56:
    print("Passed")
elif grade >= 90:
    print("Excellent!")
```

OUTPUT

```
Passed
```

Order matters!

```
grade = 95
if grade >= 56:
    print("Passed")
elif grade >= 90:
    print("Excellent!")
```

OUTPUT

```
Passed
```

⚠ A 95 that says only “Passed”

95 is also $\geq$ 56, so the first check wins.

Fix: put the strictest check first.



Decisions inside decisions

```
day = "Saturday"
raining = True
if day == "Saturday":
    if raining:
        print("Netflix and blanket")
    else:
        print("Beach day!")
else:
    print("Go to class")
```

- An `if` can live inside another `if`
- More spaces = one level deeper



Decisions inside decisions

```
day = "Saturday"
raining = True
if day == "Saturday":
    if raining:
        print("Netflix and blanket")
    else:
        print("Beach day!")
else:
    print("Go to class")
```

- An `if` can live inside another `if`
- More spaces = one level deeper

OUTPUT

```
Netflix and blanket
```



Try it!

Try it in Colab notebooks/L04/L04_01_if_elif_else.ipynb

1. Run the grade calculator with 99, 60 and 40
2. Add a level: 100 prints `"Perfect!"`
3. Write a coffee rule: over 20 ILS prints `"Make it at home"`




Predict the output

Predict the output!

```
price = 18
if price > 30:
    print("Expensive")
elif price > 15:
    print("OK")
else:
    print("Cheap")
print("Bye")
```

Predict the output

Predict the output!

```
price = 18
if price > 30:
    print("Expensive")
elif price > 15:
    print("OK")
else:
    print("Cheap")
print("Bye")
```

Answer

OK
Bye

18 > 30 is False, 18 > 15 is True. `Bye` is outside, so it always prints.

Topic 2

Combine conditions

Lecture 4 • Making Decisions



TODAY

1 if, elif, else

2 Combine conditions

3 Under the hood: 0s and 1s

Reminder: six ways to compare

Python	Means	Example (True)
==	is equal to	`5 == 5`
!=	is not equal to	`5 != 3`
< >	less / greater than	`3 < 5`
<= >=	“or equal” versions	`7 <= 7`



Every comparison is a question.
The answer: `True` or `False`.

Example: a password check

```
password = input("Password? ")
if password == "pizza123":
    print("Access granted")
else:
    print("Wrong password")
```

(And please choose a better password.)



Example: a password check

```
password = input("Password? ")
if password == "pizza123":
    print("Access granted")
else:
    print("Wrong password")
```

OUTPUT

```
Password? pizza123
Access granted
```

(And please choose a better password.)



Example: a password check

```
password = input("Password? ")
if password == "pizza123":
    print("Access granted")
else:
    print("Wrong password")
```

OUTPUT

```
Password? pizza123
Access granted
```

OUTPUT

```
Password? Pizza123
Wrong password
```

- Capital letters matter: `"P"` is not `"p"`

(And please choose a better password.)



The bouncer at the party

- Rule 1: you must be 18 **and** on the list

The bouncer at the party

- Rule 1: you must be 18 **and** on the list
- Rule 2: free entry if birthday **or** VIP

The bouncer at the party

- Rule 1: you must be 18 **and** on the list
- Rule 2: free entry if birthday **or** VIP
- Rule 3: **not** wearing flip-flops

Python has the same three words:
`and`, `or`, `not`.





and: both must be True

18+?	On the list?	Gets in?
✅	✅	
✅	❌	
❌	✅	
❌	❌	

Truth table

A table of **every** possible case and its answer.

and: both must be True

18+?	On the list?	Gets in?
✅	✅	✅
✅	❌	
❌	✅	
❌	❌	

Truth table

A table of **every** possible case and its answer.

and: both must be True

18+?	On the list?	Gets in?
✅	✅	✅
✅	❌	❌
❌	✅	❌
❌	❌	❌

Truth table

A table of **every** possible case and its answer.



and in code

L04_02_and_or_not.ipynb

```
age = 20
on_list = True
if age >= 18 and on_list:
    print("Welcome to the party!")
else:
    print("Sorry, not tonight")
```

and in code

L04_02_and_or_not.ipynb

```
age = 20
on_list = True
if age >= 18 and on_list:
    print("Welcome to the party!")
else:
    print("Sorry, not tonight")
```

OUTPUT

```
Welcome to the party!
```

and in code

L04_02_and_or_not.ipynb

```
age = 20
on_list = True
if age >= 18 and on_list:
    print("Welcome to the party!")
else:
    print("Sorry, not tonight")
```

OUTPUT

```
Welcome to the party!
```

Change `age` to 17. Now one side is `False`, so the whole thing is `False`.

or: at least one must be True

Birthday?	VIP?	Free entry?
✅	✅	
✅	❌	
❌	✅	
❌	❌	





or: at least one must be True

Birthday?	VIP?	Free entry?
✅	✅	✅
✅	❌	✅
❌	✅	✅
❌	❌	





or: at least one must be True

Birthday?	VIP?	Free entry?
✅	✅	✅
✅	❌	✅
❌	✅	✅
❌	❌	❌



Only “no and no” stays out.

or in code

```
birthday = False
vip = True
if birthday or vip:
    print("Free entry!")
else:
    print("Ticket: 50 ILS")
```

or in code

```
birthday = False
vip = True
if birthday or vip:
    print("Free entry!")
else:
    print("Ticket: 50 ILS")
```

OUTPUT

```
Free entry!
```

or in code

```
birthday = False
vip = True
if birthday or vip:
    print("Free entry!")
else:
    print("Ticket: 50 ILS")
```

OUTPUT

```
Free entry!
```

Everyday trap

In Python, `or` means “one, the other, **or both**”.

not: flip it

```
raining = False
print(not raining)
if not raining:
    print("Walk to campus")
```

raining	not raining
True	False
False	True



not: flip it

```
raining = False
print(not raining)
if not raining:
    print("Walk to campus")
```

OUTPUT

```
True
Walk to campus
```

raining	not raining
True	False
False	True



Joke break: Shakespeare, the programmer

“To be, or not to be:
that is the question.”

Joke break: Shakespeare, the programmer

“To be, or not to be:
that is the question.”

Python has the answer:

```
be = False
print(be or not be)
```

Joke break: Shakespeare, the programmer

"To be, or not to be:
that is the question."

Python has the answer:

```
be = False
print(be or not be)
```

OUTPUT

```
True
```

Try `be = True`: still `True`.

`x or not x` is **always** `True`.



Question solved.
Sorry, Hamlet.

Try it!

Try it in Colab notebooks/L04/L04_02_and_or_not.ipynb

1. Run the party code for ages 17, 18 and 30
2. Add Rule 3: `flip_flops = True` means no entry
3. Youth ticket: print `"Discount!"` if age is 18 to 30

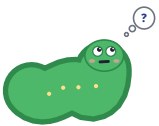




Predict the output

Predict the output!

```
x = 10
print(x > 5 and x < 8)
print(x > 5 or x < 8)
print(not x > 5)
```



Predict the output

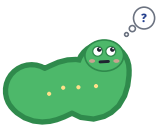
Predict the output!

```
x = 10
print(x > 5 and x < 8)
print(x > 5 or x < 8)
print(not x > 5)
```

Answer

```
False
True
False
```

`x > 5` is True, `x < 8` is False.



Topic 3

Under the hood: 0s and 1s

Lecture 4 • Making Decisions



TODAY

1 if, elif, else

2 Combine conditions

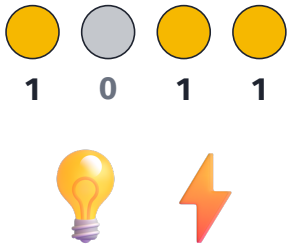
3 Under the hood: 0s and 1s

A computer is billions of tiny switches

- Each switch is **on** or **off**
- On $=$ 1 $=$ `True`, off $=$ 0 $=$ `False`

A computer is billions of tiny switches

- Each switch is **on** or **off**
- On = 1 = `True`, off = 0 = `False`
- One switch is called a **bit**
- 8 bits = 1 **byte**



Counting with only 0 and 1

- We count with 10 digits: 0 to 9
- After 9 we run out, so we write 10
- Binary has only 2 digits: 0 and 1
- After 1 it runs out, so it writes 10

Normal	Binary
0	0
1	1
2	
3	
4	
5	

Counting with only 0 and 1

- We count with 10 digits: 0 to 9
- After 9 we run out, so we write 10
- Binary has only 2 digits: 0 and 1
- After 1 it runs out, so it writes 10

Normal	Binary
0	0
1	1
2	10
3	
4	
5	

Counting with only 0 and 1

- We count with 10 digits: 0 to 9
- After 9 we run out, so we write 10
- Binary has only 2 digits: 0 and 1
- After 1 it runs out, so it writes 10

Normal	Binary
0	0
1	1
2	10
3	11
4	
5	

Counting with only 0 and 1

- We count with 10 digits: 0 to 9
- After 9 we run out, so we write 10
- Binary has only 2 digits: 0 and 1
- After 1 it runs out, so it writes 10

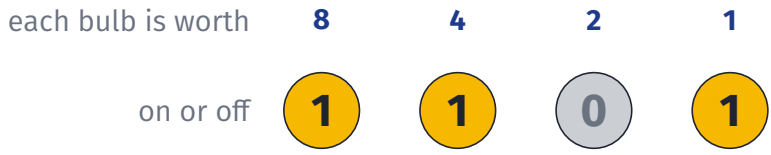
Normal	Binary
0	0
1	1
2	10
3	11
4	100
5	

Counting with only 0 and 1

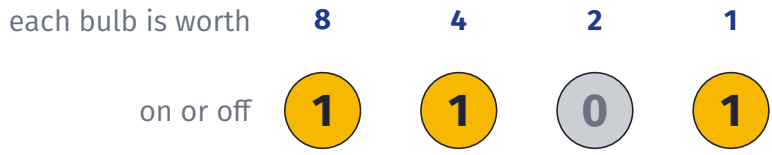
- We count with 10 digits: 0 to 9
- After 9 we run out, so we write 10
- Binary has only 2 digits: 0 and 1
- After 1 it runs out, so it writes 10

Normal	Binary
0	0
1	1
2	10
3	11
4	100
5	101

Reading a binary number

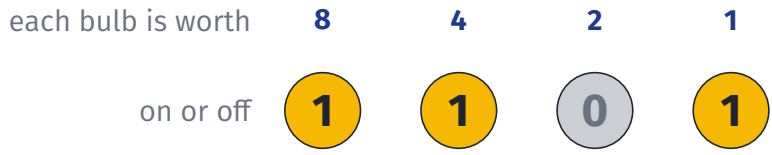


Reading a binary number



$$8 + 4 + 1 = \mathbf{13}$$

Reading a binary number



$$8 + 4 + 1 = \mathbf{13}$$

- Each place is worth **double** the one to its right
- Add up the places where the bulb is on

Joke break: 10 kinds of people

*"There are **10** kinds of people in the world:*
those who understand binary,
and those who don't."

Joke break: 10 kinds of people

*"There are **10** kinds of people in the world:*
those who understand binary,
and those who don't."




In binary, `10` is two. Now you are in the first group.

Python speaks binary: bin()

L04_03_bits_and_gates.ipynb

```
print(bin(5))
print(bin(13))
print(bin(255))
print(int("1101", 2))
```

Python speaks binary: bin()

L04_03_bits_and_gates.ipynb

```
print(bin(5))
print(bin(13))
print(bin(255))
print(int("1101", 2))
```

OUTPUT

```
0b101
0b1101
0b11111111
13
```

Python speaks binary: bin()

L04_03_bits_and_gates.ipynb

```
print(bin(5))
print(bin(13))
print(bin(255))
print(int("1101", 2))
```

OUTPUT

```
0b101
0b1101
0b11111111
13
```

- `0b` at the start just means “this is binary”
- `int("1101", 2)` goes back: binary text to a number

Logic gates: and / or as light switches

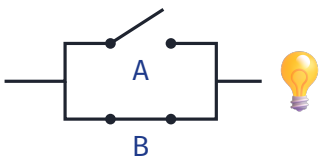
AND: switches in a row



A open: no light

Light only if A **and** B are closed.

OR: switches side by side



Light if A **or** B is closed.

Logic gates: and / or as light switches

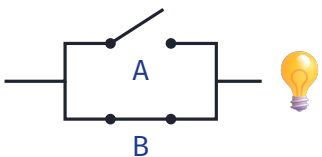
AND: switches in a row



A open: no light

Light only if A **and** B are closed.

OR: switches side by side



Light if A **or** B is closed.

Chips are built from millions of these gates.

Try it!

Try it in Colab notebooks/L04/L04_03_bits_and_gates.ipynb

1. Print your age in binary with `bin()`
2. Guess `int("1010", 2)` first, then run it
3. Print `bin(2)`, `bin(4)`, `bin(8)`. See a pattern?





Predict the output

Predict the output!

```
print(bin(6))
print(bin(8))
```

Hint: the places are worth 8, 4, 2, 1.



Predict the output

Predict the output!

```
print(bin(6))
print(bin(8))
```

Answer

```
0b110
0b1000
```

6 = 4 + 2. 8 = just the 8 place.

Hint: the places are worth 8, 4, 2, 1.



Extra: we count from 0

EXTRA

- People count 1, 2, 3, … Python counts **0, 1, 2, …**
- Coming soon: list positions start at `0` (L05), `range(4)` is `0 1 2 3` (L06)

row	count from 0	2 bits
(1)	0	00
(2)	1	01
(3)	2	10
(4)	3	11

```
print(bin(0), bin(1))
print(bin(2), bin(3))
```

OUTPUT

```
0b0 0b1
0b10 0b11
```

Count 0 to 3 with 2 bits: `00 01 10 11`. These are the 4 rows of a truth table.

Extra: a truth table is a function

EXTRA

- **Function:** inputs in, **one** output out
- **Truth table: every** input, one per row, with its output
- From now on: `0` = `False`, `1` = `True`
- **not** takes **1** input; **and**, **or** take **2**

Think about it

1. Why a table, not a sentence?
2. When is a table complete?

Extra: one input, two rows

EXTRA

	A	f(A)
(1)	0	?
(2)	1	?

Each ? can be 0 or 1.

Count them

How many **different** functions `f(A)` of one input are there?

Hint: how many choices for each row?

Extra: all the one-input functions

EXTRA

	A	f1	f2	f3	f4
(1)	0	?	?	?	?
(2)	1	?	?	?	?

Your turn

1. Fill the 4 columns, all different
2. Which one is **not** A?
3. Give the other three a name

Extra: two inputs, four rows

EXTRA

	A	B	f(A, B)
(1)	0	0	?
(2)	0	1	?
(3)	1	0	?
(4)	1	1	?

Look at the rows

1. Why exactly 4 rows?
2. Read `A B` as one binary number. What is it in row (1)? Row (4)?
3. Why this order, and not another?

Extra: how many two-input functions?

EXTRA

One function = one way to fill the four ? on the last slide.



Count them

1. How many ways to fill the 4 ?s?
2. So how many two-input functions exist?

Extra: and, or, not are just three of them

EXTRA

	A	B	A and B	A or B	not A	A == B	A != B	not (A and B)
(1)	0	0	?	?	?	?	?	?
(2)	0	1	?	?	?	?	?	?
(3)	1	0	?	?	?	?	?	?
(4)	1	1	?	?	?	?	?	?

Your turn

Fill every cell (check any row in Python, e.g. `print(True and False)`). Which two pairs of columns are exact opposites?

Extra: every function has a number

EXTRA

- Read the output column, row (1) to (4), as 4 bits
- Always `0`: `0000` $= 0$

Binary again

1. Which number is **and**? **or**?
2. Which is always 1?
3. Why only 0 to 15?

Extra: three inputs?

EXTRA

	A	B	C	f
(1)	?	?	?	?
(2)	?	?	?	?
⋮		⋮		⋮

Questions

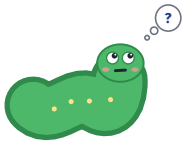
1. How many rows? Write the whole table.
2. What is in row (1)? Row (6)? The last row?
3. How many three-input functions?
4. And for n inputs: rows? functions?

Extra: think about it

EXTRA

Questions

1. Of the 16 two-input functions, how many ignore `B`?
2. How many ignore **both** inputs?
3. Build `A != B` using only **and**, **or**, **not**
4. Python has words for only 3 of the 16. Why is that enough?



Bug Zoo: the forgotten colon and indent

BUG ZOO

```
age = 20
if age >= 18
    print("Welcome")
```

ERROR

```
SyntaxError: expected ':'
```

Fix: `if age >= 18:`

```
age = 20
if age >= 18:
print("Welcome")
```

ERROR

```
IndentationError: expected an
indented block after 'if'
statement on line 2
```

Fix: push `print` 4 spaces right

AI Corner: check your conditions

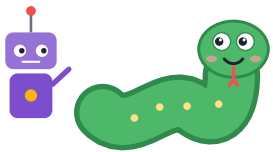
AI CORNER

A good prompt

“Here is my grade calculator. Don’t fix it. Which test grades should I try? Include edge cases like exactly 90.”

Double-check

Run each test yourself. Is 56 really “Passed”?



Recap

✔ Today you learned

- `if` / `elif` / `else`: first True wins, indentation means “belongs to”
- Compare with == != < >, combine with **`and` `or` `not`**
- Inside, everything is bits: 1 = True, 0 = False



Next time: many values in one box

- Lists: a shopping list or a playlist in Python
- Sets: no duplicates allowed
- Grab any item by its position





3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md